

Learned Metric Index in Qdrant

MSc thesis / MUNI research traineeship - supervisor briefing

Repository: /home/nicoo/work/qdrant **Branch:** thesis/lmi-integration **Evidence checkpoint:** committed Phase F baseline 6e3cfdb0f; later Phase F.2 is an uncommitted evaluation worktree. **Purpose:** oral technical/scientific briefing for Giovanni Bellitto, with evidence preserved from the Qdrant repository and saved Phase F/F.2 artifacts.

CALL CHEAT SHEET

Thesis question

Can a Learned Metric Index become a legitimate first-class physical vector index inside Qdrant while Qdrant retains ownership of storage, validity, lifecycle and exact scoring?

The broader research question is: under which data distributions, recall targets and read/update/rebuild workloads does learned routing justify its additional complexity?

One-picture architecture

BUILD

Qdrant VectorStorage + IdTracker

- > sample -> KMeans teacher -> pseudo-labels
- > Rust/tch MLP -> native MlpRouter
- > classify corpus -> bucket -> PointOffsetType postings
- > lmi_state.json -> published immutable segment

QUERY

dense query -> metric preprocessing -> MLP logits

- > top-nprobe buckets -> PointOffsetType candidates
- > Qdrant validity/deletion checks -> exact Qdrant scoring -> top-k

Current status

Implemented and tested: Qdrant-owned training, native routing, persisted learned state, reopen without training, snapshot restore, optimizer rebuild after corpus change, deletion visibility, mixed named LMI/HNSW vectors and universal/read-only LMI serving.

Experimentally demonstrated: an undertrained baseline MLP can be diagnosed and improved; selected F.2 MLPs reduce candidate work near Recall@10 0.90 on the internal F.2 holdout.

Not established: production readiness, universal superiority, HNSW replacement, a generic query planner, online learning, GPU/distributed training, or a latency advantage over HNSW.

Numbers worth remembering

Item	Recorded result
Phase F corpus	99,780 LAION/CLIP vectors, 768 dimensions, cosine, k=10
Baseline MLP vs centroid near recall 0.90	14,504 vs 10,197 candidates
Improved F.2 MLP vs centroid near recall 0.90	9,372-9,566 vs 10,231 candidates
HNSW near recall 0.90	0.16 ms native in-process p50; candidate visits unavailable
Single build observation	LMI 5.27 s/1.22 MB; HNSW 48.61 s/3.86 MB

Safe claims / unsafe claims

Safe claim: LMI is integrated as a Qdrant-owned experimental physical vector index. Qdrant owns vectors, IDs, validity, scoring, publication and lifecycle. Thirty epochs underfit the baseline. Additional training repaired most collapsed buckets. Selected models reduced candidate work around one operating point.

Do not claim: LMI is faster or better than HNSW; teacher accuracy is Recall@k; the MLP directly optimizes recall/candidate count; the selected model is globally optimal; PointOffsetType compresses vectors; Qdrant dynamically plans among all three indexes; LMI is production-ready.

Immediate next steps

- Denser nprobe sweep at matched recall, with a new external query set.
- Paired same-run centroid/affine/MLP timing and a matched-update-count sample study.
- Repeated builds, another dataset and larger scale before workload-level conclusions.

1. Motivation: vector search as a DBMS access-path problem

A vector DBMS stores high-dimensional embeddings of documents, images, audio, products or users. For a query vector q , nearest-neighbour search returns the k stored vectors most similar under a configured metric. Exact kNN compares q with every eligible vector. The work is approximately $O(Nd)$, where N is the corpus size and d is dimensionality, so exact scan becomes expensive as N grows.

ANN indexes reduce the candidate set before exact scoring. Plain is Qdrant's exact scan path; HNSW is a graph-based ANN physical access path. LMI is a learned routing access path. They belong inside a database request path:

```
request -> logical/API interpretation -> physical index
        -> candidate generation -> visibility/deletion/filter semantics
        -> metric scoring -> top-k response
```

The index is not the whole database. Correctness includes the segment's vector storage, identifier mappings, filters, deletion state and the metric implementation. There is no generic cost-based planner currently selecting Plain/HNSW/LMI per query. The physical index is configured per vector field. An LMI field uses learned routing for supported dense/default requests; `exact=true` and currently unsupported filters, non-default parameters, quantization or query forms use the implemented Plain fallback. Small/non-indexed segments can remain Plain.

2. LMI theory: teacher, router and exact scoring

For dataset D of vectors x_i in $\mathbb{R}^d$, choose sample S subset D and fit KMeans with B centers $c_1, \dots, c_B$. The teacher pseudo-label is:

$$y_i = \operatorname{argmin}_j ||x_i - c_j||^2$$

KMeans creates labels; it does not answer the final nearest-neighbour query. The router is an MLP:

```
Linear(d,h) -> ReLU -> Linear(h,B)
```

In Phase F, this was 768 -> 64 -> 64. It is trained with cross entropy on KMeans labels. The output values are logits. Softmax is unnecessary because routing needs only their ranking: top-1 assigns a corpus vector to one learned bucket, while top-nprobe sends a query to several buckets. Logits are not buckets.

The MLP approximates candidate selection, never the Qdrant metric. Candidate scoring remains exact within the selected set. The key metrics are different:

```
teacher agreement = P(MLP top-1 == KMeans label)
Recall@k(q) = |exact_top_k(q) intersection approximate_top_k(q)| / k
```

Imperfect teacher agreement can coexist with useful ANN recall because multiple buckets are probed and Qdrant exactly scores their members.

The mandatory centroid/affine control

Nearest Euclidean centroid routing is affine:

$$\begin{aligned} \operatorname{argmin}_j ||x - c_j||^2 \\ = \operatorname{argmax}_j [2 c_j^T x - ||c_j||^2] \end{aligned}$$

The term $||x||^2$ is constant over j . Therefore a nonlinear MLP is not necessary merely to reproduce Euclidean KMeans assignment. Direct centroid and affine routing are essential controls. Phase F verified identical direct/affine bucket ordering and retrieval behaviour for the tested implementation/tie rule.

3. Ownership boundary: what Qdrant owns and what LMI owns

Qdrant owns vector bytes in VectorStorage, external PointIds, the IdTracker, segment-local PointOffsetTypes, visibility/deletion, query context, distance implementation, RawScorer/BatchFilteredSearcher, top-k, snapshots, segment publication and optimizer lifecycle. LMI owns the teacher sample, router weights, bucket IDs, bucket -> PointOffsetType postings, nprobe and persisted routing metadata.

```
External PointId
-> Qdrant IdTracker
-> PointOffsetType (compact segment-local internal identifier)
-> VectorStorage row
```

LMI posting: bucket -> [PointOffsetType, ...]

PointOffsetType is not vector compression; it is an internal point reference. The postings avoid duplicating vectors. This is why the architecture is stronger than calling an external LMI library: learned routing is integrated, while DBMS semantics and final scoring remain in Qdrant.

4. Build and query pipelines

Build

```
Qdrant target segment -> enumerate eligible vectors -> seeded sample
-> CPU Lloyd KMeans -> pseudo-labels -> Rust/tch MLP
-> export weights/biases -> native MlpRouter + parity/shape validation
-> top-1 classify eligible corpus -> PointOffsetType postings
-> atomic lmi_state.json -> publish segment -> normal reopen
```

Training is database-owned. The operational path is not external Python training followed by model upload. Python/HDF5 can prepare experimental LAION vectors, but Rust/tch performs build-time training from Qdrant-owned segment storage. Local stable-Rust Lloyd KMeans replaced a dependency requiring nightly features. The model is not retrained when opened.

Query

```
q -> metric preprocessing -> native MLP forward -> B logits
-> top-nprobe buckets -> union/deduplicate candidate postings
-> Qdrant point/vector validity and context checks
-> BatchFilteredSearcher / RawScorer exact metric scoring -> top-k
```

The sentence to use orally is: **the neural network performs approximate candidate routing; Qdrant performs exact metric scoring inside the selected candidate set.** k is the requested final result size; nprobe is the number of buckets searched.

5. Segments, persistence and updates

Qdrant writes first reach mutable/appendable state. Its optimizer selects source segments, builds a new target, publishes it atomically and retires old segments. LMI belongs to an immutable indexed segment generation. When rebuild occurs after corpus change, the target VectorStorage and its PointOffsetTypes are new; LMI is retrained/reconstructed and old postings are not transplanted.

```
mutable writes -> optimizer source segments -> new immutable target
-> new storage/offsets -> new LMI state -> publication -> old segments retired
```

Restart is not rebuild. Snapshot restore is not rebuild. Open is not training. Rebuild means a changed corpus causes construction of a new indexed segment.

6. Engineering evolution

Phase	Evidence-based contribution
A (a628691f6)	Instrumented Plain/HNSW dispatch and identified physical-index selection.
B (4f3bd3798)	Added LMI structural integration; it was not yet learned ANN serving.
C	Established arbitrary PointOffsetType postings -> Qdrant-owned scoring, validity and top-k.
D	Added native Rust MLP routing and static learned postings; Torch/native inference parity work.
E (7029bcd0f)	Database-owned sampling/training/persistence/configuration through optimizer/SegmentBuilder.
E2.1 (a807381fb)	Fresh snapshot restore, no retraining on open, malformed/inconsistent state rejection.
E2 final (a1987b504)	Deferred/rebuild lifecycle, named-vector coexistence and universal/read-only LMI support.
F (6e3cfd0f)	Controlled Plain/HNSW/centroid/affine/MLP experiment.
F.2 working tree	Diagnosis and bounded training-adequacy ablation.

Phase E additionally closed configuration compilation gaps, enforced HNSW/LMI update-time exclusivity, cleaned unrelated lockfile changes and rejected unsafe persisted-state mutation. Trained state is saved in lmi_state.json because SegmentBuilder publishes and reloads segments; memory-only state would disappear.

7. Lifecycle: what is implemented and what is qualified

The accepted lifecycle evidence covers configured construction, persisted trained state, open without retraining, validation on open, fresh-storage snapshot restore, deletion validity, deferred promotion, optimizer rebuild after updates/deletes/inserts, old-state retirement, restart, mixed named LMI/HNSW fields and native/universal read-only opening.

The focused matrix passed 36 integration tests, six selected unit tests, relevant collection/segment/edge checks, default and training-enabled server builds, and isolated snapshot/lifecycle HTTP drivers. This demonstrates the accepted scope, not universal production readiness.

Remaining qualifications: no cryptographic corpus/generation binding, no append-in-place learned-state maintenance (by design), no exhaustive distributed recovery/crash injection/object-store campaign, no exhaustive quantization/query variants, limited telemetry, and no performance claim from lifecycle tests.

8. Phase F: controlled LAION baseline

The source was 100,000 x 768 float16 LAION/CLIP embeddings. Phase F indexed 99,780 float32 vectors in Qdrant, used 20 warmups and 200 disjoint held-out measured queries, cosine, k=10 and two trials per operating point. Baseline LMI: 64 buckets, sample 2,048, hidden 64, 30 epochs, batch 256, 20 Lloyd iterations, seed 42. HNSW: m=16, ef_construct=100, full_scan_threshold=0 and one indexing thread.

Timing is CPU-only, thread-controlled, in-process timing, not HTTP latency. HNSW follows native VectorIndexRead; centroid/affine/MLP use a common candidate-scoring harness. HNSW visit counts are unavailable, and ef is not a candidate-count estimate. Tiny timing differences across seams are not strong evidence.

Method/effort	Recall@10	Mean candidates	p50
Plain exact	1.0000	99,780	21.35 ms
Centroid/affine nprobe 1	.5920	2,493	.91/.89 ms
Centroid/affine nprobe 2	.7815	5,050	1.86/1.93 ms
Centroid/affine nprobe 4	.8975	10,197	3.53/3.51 ms
Centroid/affine nprobe 8	.9610	20,317	6.32/6.36 ms
Centroid/affine nprobe 16	.9925	39,228	11.37/11.20 ms
MLP nprobe 1	.5990	3,994	1.47 ms
MLP nprobe 2	.7690	7,632	2.84 ms
MLP nprobe 4	.9015	14,504	4.91 ms
MLP nprobe 8	.9650	26,468	8.16 ms
MLP nprobe 16	.9910	44,629	12.94 ms
HNSW ef16/32/64/128/256/512	.9000/.9620/.9850/.9925/.9955/.9970	unavailable	.16/.20/.32/.53/.93/1.71 ms

The baseline MLP used about 42% more candidates than centroid near recall .90. Router time was only about .06 ms at nprobe=4; candidate scoring was about 4.64 ms. The bottleneck was partition/candidate inflation, not neural forward cost. Bucket imbalance was severe: MLP 40 active/24 empty/max 7,361, versus centroid 64 active/0 empty/max 4,501.

One build measured LMI 5.27 s and 1.22 MB persisted auxiliary index, HNSW 48.61 s and 3.86 MB. Shared vector storage is excluded. This is one observation, not a general theorem of lower build cost/storage. HNSW has substantially lower measured query latency in this setup.

9. TT / MT / TM / MM: routing error decomposition

T means teacher/centroid; M means MLP. Query routing and corpus-posting assignment are independent choices. TT is teacher query + teacher postings; MT is MLP query + teacher postings; TM is teacher query + MLP postings; MM is actual MLP query + MLP postings. MT/TM are counterfactual diagnostics, not deployable indexes.

Baseline nprobe=4	Recall@10	Candidates
TT	.8975	10,196.6
MT	.8155	11,628.4
TM	.8440	11,627.0
MM	.9015	14,503.6

Changing only query routing or only corpus assignment damages consistency. Using the learned partition consistently restores roughly the original recall but inflates work. Relative to TT, MM loses about .53 exact neighbours/query and recovers .57. Candidate accounting records about 1,432 net additional candidates from changed query routing and 2,875 from changed corpus assignments, about 4,307 total. This is controlled accounting rather than unique causal attribution.

10. Phase F.2: undertraining diagnosis

All 24 baseline empty MLP classes had training examples. Their support range was 1-29, median 7, versus median 48 for active classes. They represented 10.21% of training labels and 7.00% of teacher-labelled corpus mass. Baseline sample teacher agreement was 71.73%; remaining-corpus agreement was 64.18%.

With the same 2,048 examples, fixed teacher, architecture and optimizer, varying only epochs:

Epochs	Empty buckets	Training agreement	Validation agreement
30	23-24	about 72%	55-63%
60	9-10	about 92%	76-79%

This directly supports undertraining as a major contributor to bucket collapse. It does not show every cause. At 2,048/120, remaining-corpus agreement remains 79.5-79.9%; at 16,384/60 it reaches about 86.2% with 1-2 empty buckets. Increasing samples at fixed epochs also increases optimizer updates, so coverage and optimization are confounded. 68.85% of wrong corpus assignments go to one of the four nearest teacher centroids and 83.96% to one of eight nearest: errors are often local, but this is descriptive rather than causal proof.

11. Phase F.2 selected models

The 200 Phase F queries were split into 100 validation and 100 final-test queries. Final test was withheld from F.2 selection but had already appeared in Phase F: it is an internal selection holdout, not an untouched external test set.

Stage 1: 2,048 samples; 30/60/120 epochs; seeds 42/43/44. Stage 2: selected 60 epochs; 4,096/8,192/16,384 samples; same seeds. Eighteen models were run, not a full Cartesian search. Selection used validation candidate fraction at recall targets .90/.95 within ± 0.02 ; missing coarse-grid bands received a predeclared penalty. It selected 4,096/60, not a global optimum.

Final test near Recall .90	Recall	Mean candidates
Centroid	.906	10,231.5
MLP 4096/60 seed42	.901	9,565.7
MLP 4096/60 seed43	.907	9,500.5
MLP 4096/60 seed44	.910	9,372.2

This is 6.5-8.4% lower candidate work at this operating point. Paired 2,000-resample candidate-saving intervals exclude zero for each fixed model, while recall-difference intervals include zero. Therefore the evidence supports lower candidate work on this internal test sample, not superior recall.

At roughly .95, selected MLPs use 16,464-16,824 candidates at .952-.959 recall, versus centroid 20,231 at .961. At roughly .99, centroid uses 38,856 candidates at .993; MLP nprobe32 uses 50,609-51,443 at .994-.996. MLP nprobe16 uses fewer candidates but only .976-.985 recall. The MLP does not dominate over the full recall range. F.2 timing is not directly paired with the inherited Phase F centroid timing, so no centroid latency improvement is claimed.

12. Interpreting learned routing

Cross entropy optimizes imitation of KMeans labels, not Recall@k and not candidate count. If MLP exactly copied the teacher, direct centroid/affine routing already provides the same partition. The scientifically interesting case is a learned partition that departs from the teacher but retains or improves useful neighbourhood retrieval. That is observed locally around recall .90, but it motivates future retrieval-aware objectives rather than proving they are already implemented.

13. Live Qdrant demo pipeline

The real input file was verified as /home/nicoo/work/LearnedMetricIndex/laion2B-en-clip768v2-n=100K.h5, dataset emb, shape (100000,768), dtype float16. The call demo reserves row 0 as held-out query and ingests rows 1..99999 into a 768-dimensional cosine Qdrant collection. LMI configuration: 64 buckets, sample 4096, hidden 64, epochs 60, batch 256, Lloyd 20, nprobe 4, seed 42. The optimizer threshold was raised during ingest and lowered afterward, allowing Qdrant to train internally.

Observed logs include LMI training epoch ... 60/60, followed by LMI open: mode=StaticLearned; no training. This proves the live build trained inside Qdrant and the published segment reopened persisted state without retraining. A later manual UI collection deletion is not lifecycle evidence.

Demo sequence: (1) GET collection config/status; (2) show 99,999 points/indexed vectors; (3) POST normal held-out query; (4) show candidate_source=StaticLearned candidate_count=...; (5) show top-10; (6) repeat with params.exact=true; (7) compare IDs/compute Recall@10; (8) optionally show lmi_state.json; (9) show startup/open line. Steps 3-5 prove learned routing; step 6 proves exact fallback; comparison reports approximation quality; steps 8-9 show persistence/no retraining.

14. Likely supervisor questions

Why use MLP if KMeans has buckets? KMeans/affine is the required control. MLP is tested as a learned deformation, not needed to reproduce centroid boundaries.

Teacher accuracy versus Recall@k? Teacher accuracy measures agreement with KMeans top-1 label. Recall@k measures overlap with exact Qdrant neighbours.

Why can MM recover recall with errors? Query and corpus share a learned partition; multiple buckets are searched and candidates are exactly scored.

Why are empty buckets bad? They waste output classes and concentrate postings, increasing candidate work. They are diagnostic rather than a complete causal explanation.

Why not simply use centroid routing? It is simpler and currently a strong control. The improved MLP has a local candidate-work gain but no uniform dominance.

Who computes final distance? Qdrant's existing scorer, not the MLP.

What is PointOffsetType? A segment-local internal point identifier used to find Qdrant vector rows; it is not compressed vector storage.

How are updates handled? Writes go to mutable state; optimizer rebuild creates a new immutable target and new LMI state.

Restart/snapshot? Open/restore validates and loads persisted state without retraining; both were tested in the accepted lifecycle scope.

Does Qdrant dynamically choose LMI/HNSW/Plain? No generic cost-based planner exists.

Why did HNSW look faster? Native HNSW had much lower recorded in-process latency in this setup. LMI should not be presented as its replacement.

Why 4096/60 rather than 120 epochs? Validation rule selected it; coarse powers-of-two probing penalized configurations missing a matched-recall band. This does not prove global optimality.

What remains before production? Wider datasets/scales, paired timing, workload studies, distributed/crash qualification, broader query/quantization coverage and stronger state identity validation.

15. Final oral narrative

Quick reference: status matrix

Area	Status	Careful interpretation
Qdrant construction/persistence	Implemented and tested	Experimental physical index, not production qualification
Snapshot/restart/rebuild	Implemented and tested	Accepted lifecycle scope; not exhaustive crash/distributed testing
Phase F baseline	Experimentally measured	Undertrained MLP is a preserved negative result
Phase F.2 candidate saving	Experimentally measured	Local internal-holdout result near recall .90
HNSW comparison	Experimentally measured	HNSW lower in-process latency in this setup
Future planner/online learning	Not implemented	Research direction only

60 seconds. The project asks whether learned routing can be a genuine Qdrant physical index without taking DBMS responsibilities away from Qdrant. It now builds from Qdrant vectors, persists and reopens learned state, restores snapshots and rebuilds through the optimizer. The first MLP was undertrained and worse than centroid routing in candidate work. Controlled F.2 training fixed the bucket collapse and yielded 6.5-8.4% fewer candidates around recall .90 on an internal holdout, but not a universal/high-recall/HNSW latency win. The next work is cleaner matched-recall measurement, not more features.

5-10 minute story. Start with exact scan versus ANN candidate reduction. Explain why routing must be embedded inside Qdrant rather than own a second store. Show KMeans teacher -> MLP router -> PointOffsetType postings -> Qdrant exact scorer. Emphasise the affine centroid control. Then explain Phases A-D as feasibility, E/E2 as lifecycle completion. Present the honest Phase F negative baseline: MLP had 24 empty buckets and 42% extra candidate work. Explain the F.2 intervention: more epochs removed most empty buckets, proving substantial undertraining; the validation-selected 4096/60 model reduced candidate work near one target, but not uniformly at high recall. Close with HNSW's latency advantage and the multi-objective research question: recall, candidate work, query latency, build/rebuild cost, index size and update frequency.

Evidence and reproducibility appendix

Primary evidence consulted: docs/lmi-phase-e.md, docs/lmi-phase-e2.md, docs/lmi-phase-e2-final.md, docs/lmi-phase-f-audit.md, docs/lmi-phase-f.md, docs/lmi-phase-f2.md; implementation under lib/segment/src/index/lmi_index/; Phase E/E2 tests and HTTP drivers; Phase F/F.2 evaluation code; saved JSON/CSV observations; and history at a628691f6, 4f3bd3798, 7029bcd0f, a807381fb, a1987b504, 6e3cfdb0f.

Reproducibility environment recorded by Phase F/F.2: WSL CPU, Rust 1.98, Python 3.11.16, Torch 2.5.1+cpu, tch 0.18.1 and clang 21.1.8. Exact commands, seeds, data manifests, raw observations, plots and checksums are preserved in the Phase F and Phase F.2 artifact bundles. Current worktree contains uncommitted F.2 evaluation/report files; no commit or push was performed for this briefing.